Linear Search

```
int search (int arr [], int n, int x) {
for (int i=0; i<n; i++) {
    if (arr[i] == x)
        return i;
    return -1;
}

int main (void) {
    int arr[] = {2, 3, 4, 10, 40};
    int x = 10;
    int n = sizeof (arr) / sizeof (arr[0]);
    int result = search (arr, n, x);
    (result == -1) ? printf ("Element is not Present in
                              array")
    : printf ("Element is present at index %d", result);
    return 0;
}
```

Time complexity

worst case : O(n)

Space complexity : O(1)

Binary Search

```
int binarySearch(int arr[], int n, int x) {
    int low = 0;
    int high = n-1;
    while (low <= high) {
        int mid = low + (high-low)/2;
        if (arr[mid] == x)
            return mid;
        if (arr[mid] < x)
            low = mid+1;
        else
            high = mid-1;
    }
    return -1
}
```

Time Complexity

worst Case : $O(\log n)$

Space complexity : $O(1)$

Leet Code :

1. 3658. GCD of Odd and Even Sums.

Code

```
int gcdOfOddEvenSums(int n){
    return n;
}
```

Example :

Input : n = 4
Output : 4
sumOdd = 1+3+5+7 = 16
SumEven = 2+4+6+8 = 20

Hence GCD (sumOdd, SumEven) = GCD (16, 20) = 4

For n = 4

Output : 4

2. 3867. sum of GCD of Formed Pairs.

Code :-

```
int gcd (int a, int b) {
    while (b) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
int comp (const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

long long gcdSum(int *nums, int numsSize) {
    int PrefixedGcd[numsSize];
    int maxVal = nums[0];
    for (int i = 0; i < numsSize; i++) {
        If (nums[i] > maxVal)
            maxVal = nums[i];
        PrefixGcd[i] = gcd (nums[i], maxval);
    }
    qsort (PrefixGcd, numsSize, sizeof (int), cmp);
    long long sum = 0;
    for (int i = 0; i < numsSize/2; i++) {
        sum += gcd (PrefixGcd[i], PrefixGcd
                [numsSize - 1 - i]);
    }
    return sum;
}
```

Input : nums = [2, 6, 4]
output : 2

Example :

Input : nums = [3, 6, 2, 8]
output : 5
Explanation : construct Prefix Gcd ;

i	nums[i]	mx;	PrefixGcd[i]
0	3	3	3
1	6	6	6
2	2	6	2
3	8	8	8

PrefixGcd = [3, 6, 2, 8]. After sorting, it forms [2, 3, 6, 8]
From Pairs gcd(2, 8) = 2 and gcd(3, 6) = 3, Thus, thus sum is 2+3 = 5

3. 2447. Number of Subarrays with GCD Equal to k

code :-

```
int gcd(int a, int b){
    while(b){
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int subarrayGCD(int* nums, int numsSize, int k){
    int count=0;
    for(int i=0; i<numsSize; i++){
        int g=0;
        for(int j=i; j<numsSize; j++){
            g=gcd(g,nums[j]);
            If(g==k)
                count++;
            else if(g<k)
                break;
        }
    }
    return count;
}
```

Input:
nums = [9, 3, 1, 2, 6, 3]
k = 3
Output: 4
Explanation: The subarrays of nums where 3 is the greatest common divisor of all the subarray's elements are:
[9, 3, 1, 2, 6, 3]
[9, 3, 1, 2, 6, 3]
[9, 3, 1, 2, 6, 3]
[9, 3, 1, 3, 6, 3]

4. 1998. GCD sort of an Array.

Code:

```
#define MAX 100001
int parent[MAX];
int find(int x){
    if(parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}
void unite(int a, int b){
    int pa = find(a);
    int pb = find(b);
    if(pa != pb)
        parent[pa] = pb;
}

int cmp(const void *a, constant void *b){
    return(*(int*)a - *(int*)b);
}

bool gcdSort(int* nums, int numsSize){
    for(int i=0; i<MAX; i++){
        parent[i] = i;
    int sorted[numsSize];
    for(init i=0; i<numsSize; i++)
        sorted[i] = nums[i];
    qsort(sorted, numsSize, sizeof(int), cmp);
    for(int i=0; i<numsSize; i++){
        int x = nums[i];
        for(int f=2; f*f<=x; f++){
```

```
            If(x%f==0){
                unite(nums[i],f);
                unite(nums[i],x/f);
            }
        }
    }
    for(int i=0;i<numsSize;i++){
        If(find(nums[i]!=find(sorted[i]))
            return false;
    }
    return true;
}
```

Input :
nums = [7, 21, 3]
output : true

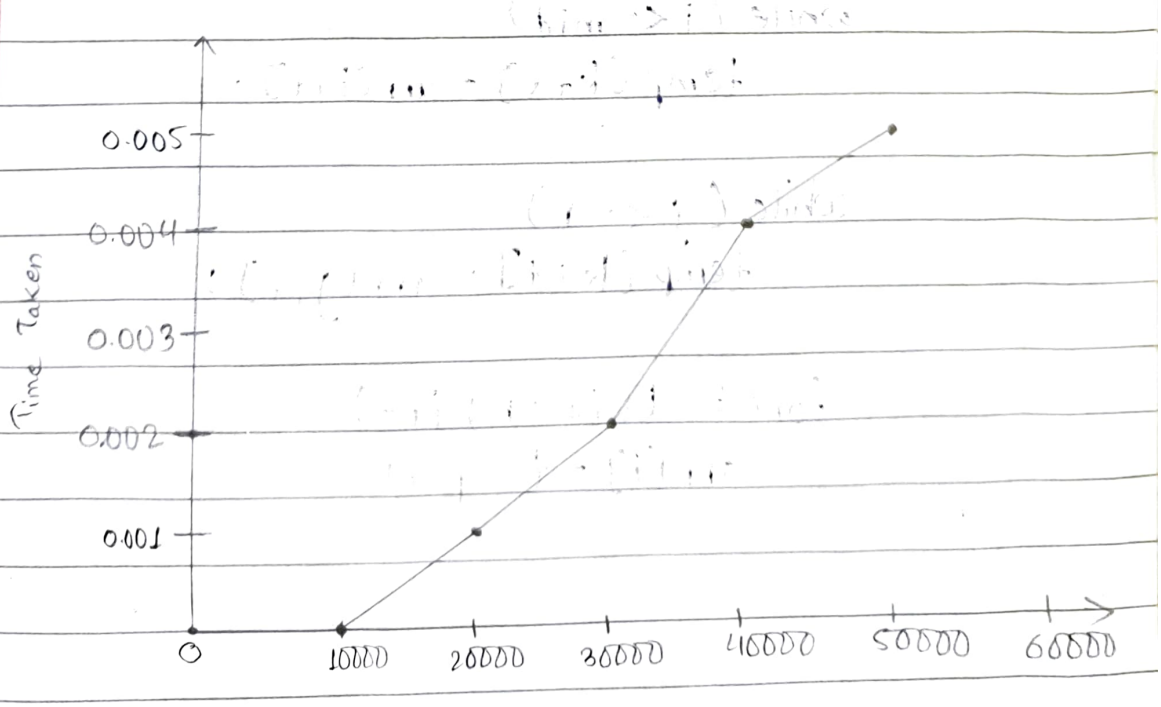
Explanation : we can sort [7, 21, 3] by performing the following operations :
swap 7 and 21 because gcd(7, 21) = 7, nums = [21, 7, 3]
swap 21 and 3 because gcd(21, 3) = 3, nums : [3, 7, 21]

Merge Sort

Pseudo Code:

```
function mergeSort (array)
    if length of array <= 1
        return array

    mid = length of array / 2

    leftHalf = array [0 ... mid]
    rightHalf = array [mid ... end]
    sortedLeft = mergeSort (leftHalf)
    sortedRight = mergeSort (rightHalf)

    return merge (sortedleft, sorted right)
```

Graph of the



Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 50000

int arr[MAX];
int temp[MAX];

void merge(int l, int mid, int r){
    int i=l, j=mid+1, k=l;
    while(i<=mid && j<=r){
        if(arr[i]<=arr[j])
            temp[k++]=arr[i++];
        else
            temp[k++]=arr[j++];
    }
    while(i<=mid)
        temp[k++]=arr[i++];

    while(j<=r)
        temp[k++]=arr[j++];

    for(i=l; i<=r; i++)
        arr[i]=temp[i];
}
```

```
void mergeSort(int l, int r){
    if(l<r){
      int mid=(l+r)/2;
      mergeSort(l, mid);
      mergeSort(mid+1,r);
      merge(l,mid,r);
    }
}

int main(){
    srand(time(NULL));
    printf("Input Size\t Time Taken(seconds)\n");
    for(int n=10000; n<=50000; n+=10000){
      for(int i=0; i<n; i++)
          arr[i]=rand()%100000;

      clock_t start = clock();
      mergesort(0,n-1);
      clock_t end = clock();
      double time_taken =(double)(end-start)/
                              CLOCKS_PER_SEC;
      printf("%d\t\t%f\n", n, time_taken);
    }
    return 0;
}
```

Output

Input Size	Time Taken (seconds)
10000	0.000
20000	0.001
30000	0.002
40000	0.004
50000	0.005

Quick sort

Input – array → A
- ~~str~~ Starting index → low
- ending index → high

Output – sorted array A

```
if low < high -
    { p ← Partition (A, low, high)
      Quick sort (A, low, p-1)
      Quick sort (A, p+1, high)
    }

Partition (A, low, high)

    pivot ← A[low]
      i ← low+1
      j ← high
      while True do -
          while i ≤ high ~~AND~~ A[i] ≤ pivot do-
              i ← i+1
          while A[j] > pivot do-
              j ← j-1

          if i < j then
              swap A[i] and A[j]
          else
              break
          end while
          ~~swap A[low] and A[j]~~
          swap A[low] and A[j]
          return j
```

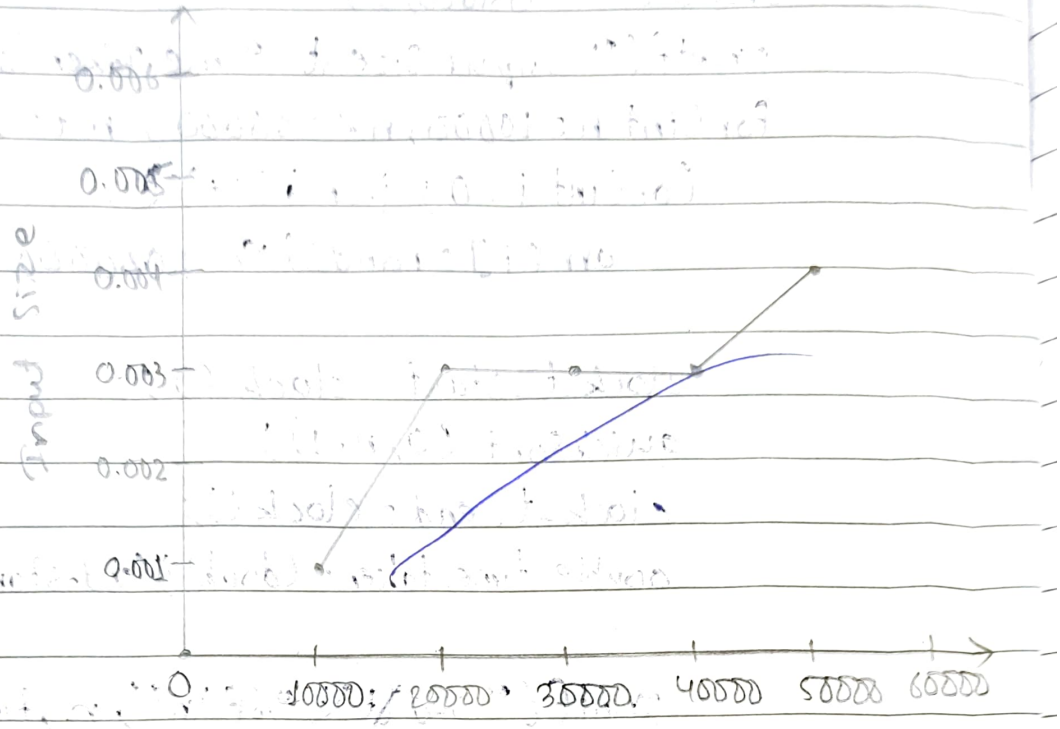
code :-

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 50000

int arr[MAX];
int partition (int low, int high) {
    int pivot = arr[high];
    int i = low-1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i+1;
}

void quickSort(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        quickSort(low, pi-1);
        quickSort(pi+1, high);
    }
}
```

```
int main() {
    srand(time(NULL));
    printf("Input Size\t Time Taken (seconds)\n");
    for(int n=10000; n<=50000; n+=10000){
        for(int i=0; i<n; i++){
            arr[i] = rand() % 100000;

        clock_t start = clock();
        quicksort(0, n-1);
        clock_t end = clock();
        double time_taken = (double)(end-start)/CLOCKS_
                                                PER_SEC;
        printf("%d\t\t %f\n", n, time_taken);
    }
    return 0;
}
```

Output

Input Size	Time Taken
10000	0.001000
20000	0.003000
30000	0.003000
40000	0.003000
50000	0.004000



Time Taken

Prims Algorithm

// Purpose : To Construct MST
// Input : A weighted connected graph
// Output : The set of edges which forms the MST

$V_T \leftarrow \{v_0\}$, $E_T \leftarrow \phi$
for $i \leftarrow 1$ to $|V|-1$ do
find an edge $e^* = (v^*, u^*)$ with minimum weight among all the edges $e = (v,u)$ such that v^* is in V_T and u^* is in $V - V_T$

$V_T \leftarrow V_T \cup \{u^*\}$
$E_T \leftarrow E_T \cup \{e^*\}$

T.C $= |E| \log |V|$

end for
return E_T

Kruskal Algorithm

Purpose, Input, output same as above.
Sort E in non-decreasing orders of the edge weights
$w(e_{i,j}) \leq \ldots\ldots \leq w(e_{i \in I})$

$E_T \leftarrow \phi$
count $\leftarrow 0$
$k \leftarrow 0$
while Count $< |V| - 1$
$k \leftarrow k+1$

If $E_T \cup \{e_{ik}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{ik}\}$

count $\leftarrow$ count + 1

end if

end while

return E_T

$T_{KC} = O(|E| \log |E|)$

Find the minimum cost spanning of a given undirected graph using prims Algorithm.

Code :-

```
# include <stdio.h>
# define INF 999

int main() {
    int n, i, j;
    int cost[10][10], visited[10] = {0};
    int min, a, b, u, v;
    int ne = 1, mincost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix :\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    visited[1] = 1;
    printf("\nEdges in MST :\n");
    while (ne < n) {
        min = INF;
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                if (cost[i][j] < min) {
                    if (visited[i] != 0) {
```

```
            min = cost[i][j];
            a = u = i;
            b = v = j;
        }
    }
}
if(visited[u]==0 || visited[v]==0){
    printf("Edge %d : (%d %d) cost = %d\n",
            ne++, a, b, min);
    mincost += min;
    visited[b] = 1
}
cost[a][b] = cost[b][a] = INF;
}
printf("\n Minimum cost = %d \n", mincost);
return 0;
}
```

Output:

```
Enter number of vertices : 4

Enter cost adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Edge in MST
0 -> 1   cost = 2
1 -> 2   cost = 3
0 -> 3   cost = 6

Total cost of MST = 11
```

Find minimum cost spanning tree of a given undirected graph using Kruskal's Algorithm.

Code :-

```
#include <stdio.h>
#define MAX 100

int parent[MAX];

int Find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionset(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

int main() {
    int i, j, ne = 0;
    int cost[MAX][MAX];
    int min, u, v, a, b;
    int totalCost = 0;
    printf("Enter number of vertices : ");
    scanf("%d", &n);
    printf("Enter cost-adjacency matrix : \n");
    for(i=0; i<n; i++) {
```

```
        for (j=0; j<n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = 9999;
        }
    }
    for (i=0; i<n; i++)
        parent[i] = i;
    printf("\nEdges in MST:\n");

    while (ne < n-1) {
        min = 9999;

        for (i=0; i<n; i++) {
            for (j=0; j<n; j++) {
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    u = a = i;
                    v = b = j;
                }
            }
        }
        if (find(a) != find(b)) {
            printf("%d => %d cost = %d\n", u, v, min);
            totalcost += min;
            unionset(a, b);
            ne++;
        }
        cost[u][v] = cost[v][u] = 9999;
    }
    printf("\n Total cost of MST = %d\n");
    return 0;
}
```

Output

Enter number of vertices : 5

Enter cost adjacency matrix :
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in MST :
0 → 1 cost = 2
1 → 2 cost = 3
1 → 4 cost = 5
0 → 3 cost = 6

Total cost of MST = 16

TOPOLOGICAL_SORT

Algorithm :-

//Purpose : TOPOLOGICAL SORT
//Input : A directed graph G=(V,E)
//Output : A topological ordering of vertices

Compute indegree [V] for all v∈V
Create an empty queue Q
for each vertex v∈V do
    if indegree[v] = 0 then
        ENQUEUE (Q, V)
count ← 0

while Q is not empty do
    v ← DEQUEUE (Q)
    print v
    count ← count + 1
    for each adjacent vertex u of v do
        indegree[u] ← indegree[u] - 1
        if indegree[u] ← indegree[u] = 0 then
            ENQUEUE (Q, u)

if count ≠ |V| then
    print "Cycle exists (No Topological Order)"
end

Code!

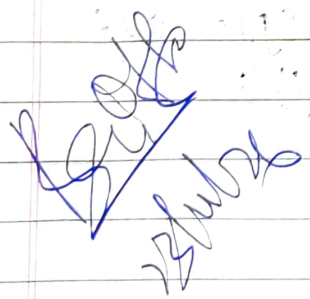
```
#include <stdio.h>
#define MAX 100

int queue[MAX], front=-1, rear=-1;
void enqueue(int x) {
    if (rear == MAX-1) return;
    if (front == -1) front = 0;
    queue[++rear] = x;
}
int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}

int isEmpty() {
    return (front == -1 || front > rear);
}

int main() {
    int n, i, j;
    int adj[MAX][MAX], indegree[MAX]={0};
    int topo[MAX], count=0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i=0; i<n; i++) {
        for(j=0; j<n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
```

```
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        if (adj[i][j] == 1) {
            indegree[j]++;
        }
    }
}
for (i=0; i<n; i++) {
    if (indegree[i] == 0) {
        enqueue(i);
    }
}

while (!isEmpty()) {
    int v = dequeue();
    topo[count++] = v;
    for (j=0; j<n; j++) {
        if (adj[v][j] == 1) {
            indegree[j]--;
            if (indegree[j] == 0) {
                enqueue(j);
            }
        }
    }
}
if (count != n) {
    printf("Cycle detected! Topological sort
            not possible.\n")
}
else {
    printf("Topological order:\n");
    for(i=0; i<n; i++) {
```

```
        printf("%d ", topo[i]);
    }
  }
  return 0;
}
```

Output:

```
Enter number of vertices: 4
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological order:
0 2 1 3
```



JOHNSON TROTTER

Algorithm :

```
DEFINE LEFT = -1
DEFINE RIGHT = 1

FUNCTION getLargestMobile (perm, dir, n);
    largestMobile ← 0
    index ← -1
    FOR i FROM 0 TO n-1 :
        neighbor ← i + dir[i]
        IF neighbor is within bounds AND perm[i]
                > perm[neighbor]:
            IF perm[i] > largestMobile :
                largestMobile ← perm[i]
                index ← i

    RETURN index
END FUNCTION

FUNCTION printPermutation (perm, n):
    FOR i FROM 0 TO n-1 :
        PRINT perm[i]
    PRINT newline
END FUNCTION

FUNCTION johnson Trotter (n);
    DECLARE perm[n], dir[n]
    FOR i FROM 0 TO n-1 :
    perm[i] ← i + 1
    dir[i] ← LEFT
```

~~Code~~

```
~~#include <stdio.h>~~
~~#define LEFT -1~~
~~#define RIGHT 1~~

~~int getLargestMobile(int perm[], int~~

CALL printPermutation(perm, n)

WHILE TRUE:
    largestIndex ← getLargestMobile(perm, dir, n)
    IF largestIndex == -1;
    ~~BREAK~~ BREAK
    swapIndex ← ~~getLargest~~largestIndex + dir[largest
                                                Index]

    swap perm[largestIndex] WITH perm
                                [swapIndex]
    swap dir[largestIndex] WITH dir[swapIndex]

    Reverse directions of large elements.
```

Code:

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

int getLargestMobile(int perm[], int dir[], int n) {
    int LargestMobile = 0, index = -1;
    for (int i = 0; i < n; i++) {
        int neighbor = i + dir[i];
        if (neighbor >= 0 && neighbor < n && perm[i]
                > perm[neighbor]) {
            if (perm[i] > LargestMobile) {
                largestMobile = perm[i];
                index = i;
            }
        }
    }
    return index;
}

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", perm[i]);
    printf("\n");
}

void johnsonTrotter(int n) {
    int perm[n], dir[n];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
```

```
printPermutation(perm, n);
while (1) {
    int largestIndex = getLargestMobile(perm, dir, n);
    if (largestIndex == -1)
        break;
    int swapIndex = largestIndex + dir[largestIndex];
    int temp = perm[largestIndex];
    perm[largestIndex] = perm[swapIndex];
    perm[swapIndex] = temp;
    int tempDir = dir[largestIndex];
    dir[largestIndex] = dir[swapIndex];
    dir[swapIndex] = tempDir;
    for (int i = 0; i < n; i++)
        if (perm[i] > perm[swapIndex])
            dir[i] = -dir[i];
    printPermutation(perm, n);
  }
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("All permutations:\n");
    johnsonTrotter(n);
    return 0;
}
```

Output :

Enter n : 3

All permutations :

1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Heap Sort

Algorithm:

```
function heapify (arr [], n, i, type):
    extreme = i
    left = 2 * i + 1
    right = 2 * i + 2
    if type == 1:
        if left < n and arr [left] > arr [extreme
            extreme = left
        if right < n and arr [right] > arr [extreme
            extreme = right
    else:
        if left < n and arr [left] < arr [extreme
            extreme = left
        if right < n and arr [right] < arr [extreme
            extreme = right

    if extreme != i:
        swap arr [i] with arr [extreme]
        heapify (arr, n, extreme, type)

function buildHeap (arr [], n, type):
    for i from n/2 - 1 down to 0:
        heapify (arr, n, i, type)

function heapSort (arr [], n, type):
    buildHeap (arr, n, type)
    for i from n-1 down to 1:
        swap arr [0] with arr [i]
        heapify (arr, i, 0, type)
```

Code :

```
#include <stdio.h>

void heapify (int arr[], int n, int i, int type) {
    int extreme = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if (type == 1) {
        if (left < n && arr[left] > arr[extreme])
            extreme = left;
        if (right < n && arr[right] > arr[extreme])
            extreme = right;
    }
    else {
        if (left < n && arr[left] < arr[extreme])
            extreme = left;
        if (right < n && arr[right] < arr[extreme])
            extreme = right
    }
    if (extreme != i) {
        int temp = arr[i];
        arr[i] = arr[extreme];
        arr[extreme = temp;
        heapify (arr, n, extreme, type);
    }
}

void buildHeap (int arr[], int n, int type) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify (arr, n, i, type);
}
```

```
void heapSort(int arr[], int n, int type){
    buildHeap(arr, n, type);
    for(int i=n-1; i>0; i--){
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0, type);
    }
}

void printArray(int arr[], int n){
    for(int i=0; i<n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main(){
    int arr1[] = {4, 10, 3, 5, 1};
    int arr2[] = {4, 10, 3, 5, 1};
    int n=5;
    heapSort(arr1, n, 1);
    printf("Ascending (Max Heap): \n");
    printArray(arr1, n);
    heapSort(arr2, n, 0);
    printf("Descending (Min Heap): \n");
    printArray(arr2, n);
    return 0;
}
```

Output:

Ascending (Max Heap):
1 3 4 5 10

Descending (Min Heap):
10 5 4 3 1

Rohit
Tawde.

Floyd Algorithm

Code:

```
#include<stdio.h>
#define INF 99999
#define MAX 10

void floydwarshall(int n, int graph[MAX][MAX]){
int dist[MAX][MAX];
for(int i=0;i<n;i++){
   for(int j=0;j<n;j++)
      dist[i][j]=graph[i][j];
      for(int k=0;k<n;k++){
for(int k=0; k<n; k++){
   for(int i=0; i<n; i++){
      for(int j=0; i<n; j++){
         if(dist[i][k]!=INF&&dist[k][j]!=INF&&
            dist[i][k]+dist[k][j]<dist[i][j]){
            dist[i][j]=dist[i][k]+dist[k][j];
         }
      }
   }
}
for(int i=0;i<n;i++){
   if(dist[i][i]<0){
      printf("Negative cycle detected!\n");
      return;
   }
}
printf("\n All-pairs Shortest Path matrix:\n");
for(int i=0;i<n;i++){
   for(int j=0;j<n;j++){
```

```
            if(dist[i][j] == INF)
                printf("INF");
            else
                printf("%d", dist[i][j]);
        }
    }
    printf("\n");
}
}
int main(){
int n;
int graph[MAX][MAX];
printf("Enter adjacency matrix (use %d for INF):\n", INF);
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        scanf("%d", &graph[i][j]);
    }
}
floydWarshall(n, graph);
return 0;
}
```

O/p:

```
Enter number of vertices: 5
Enter adjacency matrix (use 99999 for INF):
  0     4     99999   5     99999
99999   0     1     99999   6
  2   99999   0     3     99999
99999 99999   1     0     2
  1   99999 99999   4     0
```

All pairs shortest path matrix:

```
0  4  5  5  7
3  0  1  4  6
2  6  0  3  5
3  7  1  0  2
1  5  5  4  0
```

LeetCode:1334 → Find the city with the smallest number of Neighbors at a Threshold Distance

Code:

```
#include <stdio.h>
#include <limits.h>
#define INF 1000000000
int FindTheCity(int n, int **edges, int edgesSize, int *edgesColSize, int distanceThreshold){
int dist[n][n];
for(int i=0; i<n; i++){
for(int j=0; j<n; j++){
    if(i==j)
        dist[i][i]=0;
    else
        dist[i][j]=INF;
}
}
for(int i=0; i<edgesSize; i++){
    int u=edges[i][0];
    int v=edges[i][1];
    int w=edges[i][2];
    dist[u][v]=w;
    dist[v][u]=w;
}
for(int k=0; k<n; k++){
    for(int i=0; i<n; i++){
        for(int j=0; i<n; j++){
            if(dist[i][k]+dist[k][j]<dist[i][j]){
                dist[i][j]=dist[i][k]+dist[k][j];
            }
        }
    }
}
```

```
int result = -1;
int mincount = INF_MAX;
for (int i=0; i<n; i++) {
    int count = 0;
    for (int j=0; j<n; j++){
        if (i != j && distance [i][j] <= distance
            Threshold) {
            count ++;
        }
    }
    if (count<=mincount){
        mincount = count;
        result = i;
    }
}
return result;
}
```

Output:

case-1

Input

n=4

edges = [{0,1,3}, {1,2,1}, {1,3,4}, {2,3,1}]

distance Threshold = 4

output = 3

Expected = 3.

case-2

Input n=5

edges = [{0,1,2}, {0,4,8}, {1,2,3}, {1,4,2}, {2,3,1}, {3,4,1}]

distanceThreshold = 2

Output = 0 Expected = 0

LeetCode: 207 → Course Schedule

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define WHITE 0
#define GRAY 1
#define BLACK 2

bool dfs(int node, int n, int **graph, int *color){
    color[node] = GRAY;
    for(int i=0; i<n; i++){
        if(graph[node][i]){
            if(color[i]==GRAY) return true;
            if(color[i]==WHITE && dfs(i,n,graph,
                                      color))
                return true;
        }
    }
    color[node]=BLACK;
    return False;
}
bool canFinish(int numCourses, int *prerequisitor; int
               prerequitisesize, int prerequsitercolsize){
    int canfinish(int
    int n = numcourses;
    int **graph = (int **graph = (int **)malloc(n*sizeof
                                        (int*)));
    for(int i=0; i<n; i++){
        graph[i]=(int*) calloc(n, sizeof(int));
    }
```

```
for(int i=0; i<prerequisitesize, i++){
    int u = prerequisites[i][1];
    int v = prerequisites[i][0];
    graph[u][v] = 1;
}
int *color = (int*) calloc(n, sizeof(int));
for(int i=0; i<n; i++){
    if (color[i] == WHITE){
        if(dfs(i, n, graph. color))
            return false;
    }
}
return
}
```

Output:

case 1:

num courses = 2

prerequisites = [[1,0],[0,1]]

output : False

Expected : False.

case 2:

num courses = 2

prerequisites: [[1,0]]

output : True

Expected : True

Leetcode: 912 ⇒ sort an Array:

code:

```
#include <stdio.h>
#include <stdlib.h>

void merge(int* nums, int left, int mid, int right){
    int n1 = mid-left+1;
    int n2 = right-mid;
    int *L = (int *)malloc(n1 *sizeof(int));
    int* R = (int *)malloc(n2 *sizeof(int));
    for(int i=0; i<n1; i++)
        L[i] = nums[left+i];
    for(int j=0; j<n2; j++)
        R[j] = nums[mid+1+j];
    int i=0, j=0, k=left;
    while(i<n1 && j<n2){
        if(L[i] <= R[j])
            nums[k++] = L[i++];
        else
            nums[k++] = R[j++];
    }
    while(i<n1)
        nums[k++] = L[i++];
        while(i<n2)
            nums[k++] = R[i++];
        free[L];
        free[R];
    }
}
void mergesort[int* nums, int left, int right){
    if(left >= right) return;
```

```
    int mid = left+(right-left)/2;
    mergesort(nums, left, mid);
    mergesort(nums, mid+1, right);
    mergesort(nums, left, mid, right);
}
int *sortArray(int **nums, int numssize,
               int *returnsize){
mergesort[norms, 0, numssize-1);
*returnsize=numssize;
return nums;
}
```

Output :

case 1 :

nums = [5, 2, 3, -]

output : [1, 2, 3, 5]

Expect [1, 2, 3, 5]

case 2 :

nums = [5, 1, 1, 2, 0, 0]

output = [0, 0, 1, 1, 2, 5]

Expected [0, 0, 1, 1, 2, 5]

0/1 KNAPSACK (Dynamic Programming)

Code!

```
#include <stdio.h>

int max(int a, int b) {
    if (a>b)
        return a;
    else
        return b;
}
int knapsack (int W, int wt[], int val[], int n){
    int i, w;
    int K[n+1] [W+1];
    for (i=0; i<=n; i++) {
        for(w=0; w<=W; w++) {
            if (i==0 || w==0){
                K[i][w]=0;
            }
            else if (wt[i-1]<=w) {
                K[i][w] = max(val[i-1]+K[i-1]
                [w-wt[i-1]], K[i-1][w]);
            }
            else {
                K[i][w] = k[i-1][w];
            }
        }
    }
    return K[n][w];
}
```

```
int main(){
    int n, W, i;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    for (i=0; i<n; i++){
        printf("\n Enter value of item %d :", i+1);
        scanf("%d", &val[i]);
        printf("Enter weight of item %d : ", i+1);
        scanf("%d", &wt[i]);
    }
    printf("\n Enter capacity of knapsack: ");
    scanf("%d", &W);
    printf("\n Maximum Profit = %d \n", knapsack
                                    (W, wt, val, n));
    return 0;
}
```

Output:

```
Enter number of items: 3
Enter value of item 1: 20
Enter weight of item 1: 3

Enter value of item 2: 40
Enter weight of item 2: 1

Enter value of item 3: 30
Enter weight of item 3: 2

Enter capacity of knapsack: 4
Maximum Profit = 70
```

Pseudo Code

```
KNAPSACK (W, wt[], val[], n)
Create table K[0....n][0....W]
for i = 0 to n
    for w = 0 to W
        if i==0 OR w==0
            K[i][W] = 0
        else if wt[i-1] <= w
            K[i][w] = max (val[i-1] + k[i-1][w-wt[i-1]],
                           K[i-1][w])
        else
            K[i][w] = K[i-1][w]
return K[n][W]
```

Leetcode: 494 → Target Sum (0/1 Knapsack DP approach)

Code:

```
int findTargetSumWays(int* nums, int numsSize, int target){
    int sum=0;
    for(int i=0; i<numsSize; i++){
        sum+=nums[i];
    }
    if((sum+target)%2!=0||abs(target)>sum){
        return 0;
    }
    int subsetSum=(sum+target)/2;
    int dp[subsetSum+1];

    for(int i=0; i<=subsetSum; i++){
        dp[i]=0;
    }
    dp[0]=1;
    for(int i=0; i<numsSize; i++){
        for(int j=subsetSum; j>=nums[i]; j--){
            dp[j]+=dp[j-nums[i]];
        }
    }
    return dp[subsetSum];
}
```

Output

nums = [1,1,1,1,1] } Input
target = 3

Output = 5
expected = 5

Fractional Knapsack

Code:

```
#include <stdio.h>

struct Item {
    int weight;
    int value;
    float ratio;
};

void swap (struct Item *a, struct Item *b) {
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}

void sortItems (struct Item items[], int n) {
    for (int i=0; i<n-1; i++) {
        for (int j=0; j<n-i-1; j++) {
            if (items[j].ratio < items[j+1].ratio) {
                swap (&items[j], &items[j+1]);
            }
        }
    }
}

float fractionalKnapsack (int capacity, struct Item items[],
                          int n) {
    sortItems (items, n);
    float totalValue = 0.0;
```

```
    For (int i=0; i<n; i++) {
        if (capacity >= items[i].weight) {
            capacity -= items[i].weight;
            totalValue += items[i].value;
        }
        else {
            totalValue += items[i].value * ((float)
                                capacity / items[i].weight);
            break;
        }
    }
    return totalValue;
}

int main() {
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item items[n];
    for (int i=0; i<n; i++) {
        printf("Enter value and weight of item %d: ",
                i+1);
        scanf("%d %d", &items[i].value, &items[i].
                weight);
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    float maxValue = fractionalKnapsack(capacity, items, n);
    printf("Maximum value in knapsack = %.2f \n", maxval);
    return 0;
}
```

Output

```
Enter number of items : 3
Enter value and weight of item 1 : 10  3
Enter value and weight of item 2 : 30  2
Enter value and weight of item 3 : 40  2
Enter knapsack capacity : 4
Maximum value in knapsack = 70.00
```

Pseudo code :

```
Input items with value and weight
For each item :
    Calculate ratio = value / weight
Sort items in descending order of ratio
total Value = 0

For each item :
    If capacity >= item weight :
        Take full item
        capacity = capacity - item weight
        totalvalue = total Value + item value
    Else :
        Take fractional part
        totalValue += item value * (capacity / item weig
        Break
Return totalValue
```

LeetCode:316 → Remove Duplicate letters

code:

```
char * removeDuplicateLetters(char *s){
    int last[26], top=-1, n=strlen(s);
    bool visited[26]={0};
    for(int i=0; i<n; i++)
        last[s[i]-'a']=i;

    char *st=malloc(n+1);

    for(int i=0; i<n; i++){
        char c=s[i];
        if(vis[c-'a'])continue;
        while(top>=0 && c<st[top] && i<last[st
                                    [top]-'a']){
            vis[st[top--]-'a']=0;
        }
        st[++top]=c;
        vis[c-'a']=1;
    }
    st[top+1]='\0';
    return st;
}
```

Input
s = "bcabc"

Output = "abc"

Expected = "abc"

Dijkstra's Algorithm

Code :-

```
#include <stdio.h>
#define MAX 10
#define INF 99999
int minDistance(int dist[], int visited[], int n){
    int min = INF, min_index = -1;
    for(int i = 0; i < n; i++){
        if(!visited[i] && dist[i] <= min){
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}
void dijkstra(int graph[MAX][MAX], int n, int src){
    int dist[MAX], visited[MAX];
    for(int i=0; i<n; i++){
        dist[i] = INF;
        visited[i] = 0;
    }  dist[src] = 0;

    for(int count = 0; count < n-1; count++){
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
        for(int v=0; v<n; v++){
            if(!visited[v] && graph[u][v] && dist[u]
                != INF && dist[u] + graph[u][v]
                < dist[v]){
                dist[v] = dist[u] + graph[u][v];
            }
        }
```

```
    }
  }
  printf("\n Vertex \t Distance from Source\n");
  for (int i=0; i<n; i++) {
      printf("%d \t %d\n", i, dist[i]);
  }
}
int main() {
  int n=5;
  int graph[MAX][MAX] = {
      {0, 10, 0, 5, 0},
      {0, 0, 1, 2, 0},
      {0, 0, 0, 0, 4},
      {0, 3, 9, 0, 2},
      {7, 0, 6, 0, 0}
  };
  int source = 0;
  dijkstra(graph, n, source);
  return 0;
}
```

Output :-

vertex	distance from source
0	0
1	8
2	9
3	5
4	7

N Queens

Code :-

```
#include <stdio.h>
#include <stdlib.h>

int N;
int *board;

int isSafe (int row, int col){
    for (int i=0; i<row; i++){
        if (board[i]==col || abs(board[i]-col)==
                abs(i-row)){
            return 0;
        }
    }
    return 1;
}

void printSolution(){
    for (int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            if (board[i]==j)
                printf("Q");
            else
                printf(".");
        }
        printf("\n");
    }
    printf("\n");
}
```

```
void solveNQueens(int row){
    if (row == N) {
        printSolution();
        return;
    }
    for (int col=0; col<N; col++) {
        if (isSafe(row, col)) {
            board[row]=col;
            solveNQueens(row+1);
        }
    }
}
int main(){
    printf("Enter the value of N:");
    scanf("%d", &N);
    board = (int*) malloc(N * sizeof(int));
    printf("\nSolutions for %d-Queens Problem:\n\n"
            N);
    solveNQueens(0);
    free(board);
    return 0;
}
```

Output:

Enter the value of N: 4

. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .